Doctoral Program in Engineering Sciences at ITESO

VEHICLE DYNAMICS AND CONTROL SIMULATION PLAYGROUND: THE USER MANUAL

L. Arturo Torres-Romero and, Luis E. González-Jiménez

April 8, 2026
Tlaquepaque, Jalisco, México, 45604

Doctoral Program in Engineering Sciences
ITESO *(Instituto Tecnológico y de Estudios Superiores de Occidente)*

©L. Arturo Torres-Romero, Luis E. González-Jiménez

No part of this document may be copied, translated, transcribed or entered in any form into any machine without written permission. Address inquires in this regard to the authors or to the Chair of the Doctoral Program in Engineering Sciences at ITESO (dci@iteso.mx). Excerpts may be quoted for scholarly purposes with full acknowledgment of source. This document may not be lent or circulated without this cover page.

VEHICLE DYNAMICS AND CONTROL SIMULATION PLAYGROUND: THE USER MANUAL

L. Arturo Torres-Romero, Luis E. González-Jiménez

April 8, 2026

Doctoral Program in Engineering Sciences
ITESO *(Instituto Tecnológico y de Estudios Superiores de Occidente)*

Tlaquepaque, Jalisco, México, 45604
Tel +52 33 3669 3598

E-mail: dci@iteso.mx

Keywords *DC motor, BLDC, electric motor, dq model, Park-Clarke, Clarke, PID,lateral dynamics, electric vehicle*

Abstract

This user manual provides documentation for a Simulink and MATLAB model designed to support students and researchers in the fields of vehicle dynamics, control systems, and electric motor applications. The model is publicly available to facilitate learning, experimentation, and validation of custom models or control strategies. By sharing this resource, our aim is to contribute to the advancement of clean and intelligent mobility, particularly in the development of electric and autonomous vehicles.

I. INTRODUCTION

This project consists of a MATLAB/Simulink model designed to simulate the longitudinal and lateral dynamics of a vehicle, incorporating electric motors as the primary actuators. The main objective is to provide a flexible and modular environment for the study and development of control strategies for electric and autonomous vehicles.

At first encounter, vehicle dynamics and the control theory applied to it may appear challenging. The mathematical formulation and the variety of physical models involved can be difficult to fully grasp at an initial stage. However, this is a natural part of the learning process. As familiarity increases, the subject reveals itself as both engaging and intellectually rewarding.

This work was supported in part by CONACYT (*Consejo Nacional de Ciencia y Tecnología*, Mexican Government) through the recognition granted to the PhD Program in Engineering Sciences at ITESO and Continental Automotive Guadalajara S.A. de C.V. through a scholarship granted to L. Arturo Torres-Romero.

It is important to note that this model does not aim to represent a complete or exhaustive description of vehicle behavior. At the time of writing, it should be regarded as an introductory framework. The intention is to progressively extend its capabilities by incorporating additional dynamics and improving its level of fidelity. Nevertheless, it is also worth considering that different levels of model complexity may be required depending on the application. In some cases, simplified models may be more appropriate than highly detailed ones. The most suitable approach will be evaluated over time.

At its current stage, the model is intended to facilitate the understanding of the fundamental dynamics governing vehicle motion. In particular, it focuses on the physical forces and phenomena acting on a vehicle as it moves from one point to another. It is worth emphasizing that a vehicle, typically with a mass on the order of one ton or more, interacts continuously with the road through complex mechanisms that are not always immediately apparent.

We begin with a set of fundamental definitions and concepts:

Vehicle Dynamics: Refers to the study of how vehicles respond to driver inputs, road conditions, and external forces. It encompasses multiple subsystems, including longitudinal, lateral, and tire dynamics. Figure 1 illustrates, on the left, the longitudinal dynamics and, on the right, the lateral dynamics, which are described below.

Longitudinal Dynamics: Describes the motion of the vehicle along the forward and reverse directions. It includes acceleration, braking, and traction phenomena. The main influencing factors are engine or motor power, braking forces, rolling resistance, and aerodynamic drag.

Lateral Dynamics: Governs the side-to-side motion of the vehicle, including yaw and lateral displacement. It determines steering response, cornering behavior, and overall stability. Key factors include steering angle, tire slip angle, and centrifugal effects.

Tire Dynamics: Describes the mechanisms through which tires generate forces and grip on the road surface. This includes slip behavior, friction characteristics, and deformation under load. These effects are commonly modeled using formulations such as the Pacejka "Magic Formula". Together, these dynamics define the handling, stability, and safety of the vehicle under various driving conditions.

In the context of autonomous driving, the vehicle is required to follow a predefined reference path. A widely adopted approach, popularized during the DARPA Grand Challenge (2004, 2005) and the Urban Challenge (2007), is the use of **waypoints**. Waypoints are predefined reference points used to guide a vehicle along a desired trajectory.

In autonomous vehicle applications, waypoints represent positions in two- or three-dimensional

space that the vehicle follows to reach its destination. They can be categorized as follows:

Global Waypoints: Represent large-scale path planning references, typically obtained from GPS-based maps. They define the overall route from the starting point to the final destination.

Local Waypoints: Correspond to short-range reference points used for fine motion control and path smoothing. These are often generated dynamically based on sensor information and current road conditions.

Each waypoint typically contains the following information:

- Position (x, y, z): Defines the location in a global coordinate frame.
- Orientation (yaw, pitch, roll): Specifies the heading and attitude of the vehicle.
- Velocity or time stamp: Used for trajectory generation and motion planning.

.

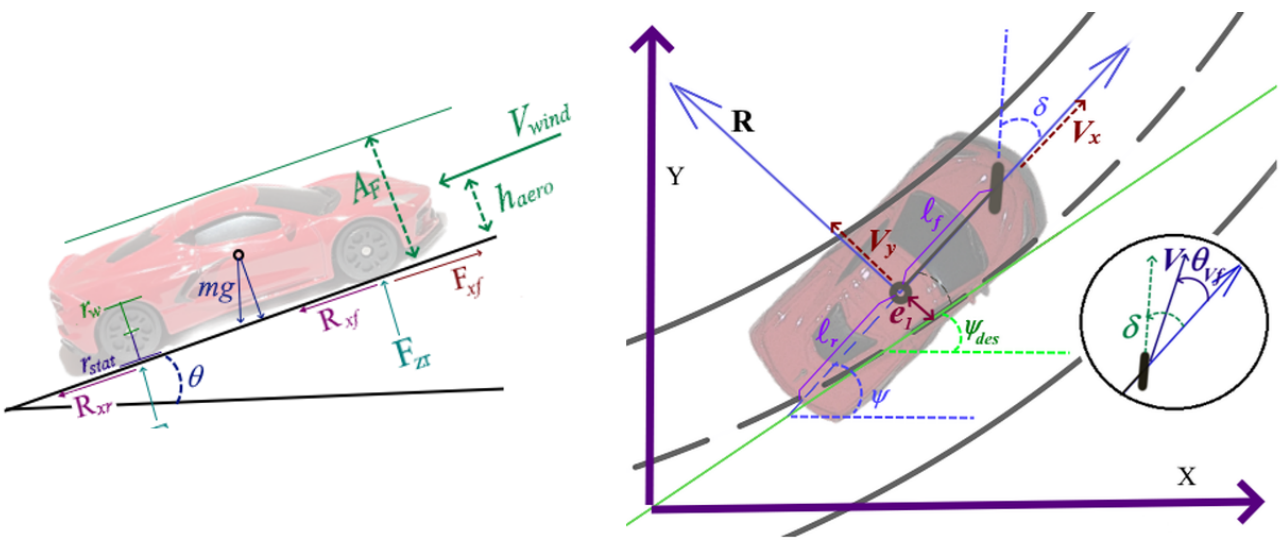

Figure 1: Typical scheme of the longitudinal and lateral dynamics

II. QUICK START

The intention of this section is to guide you to perform the first simulation of the vehicle and show you the first insights you can get at first hand.

Download the repo in Github from [1]. Fig. 2 shows the root directory of the repository. For now, we will be interested only in the highlighted directory named **./VehicleModel**. Go inside. In Fig. 3 the files within are shown. the marked files are the ones we are interested for the moment. The most

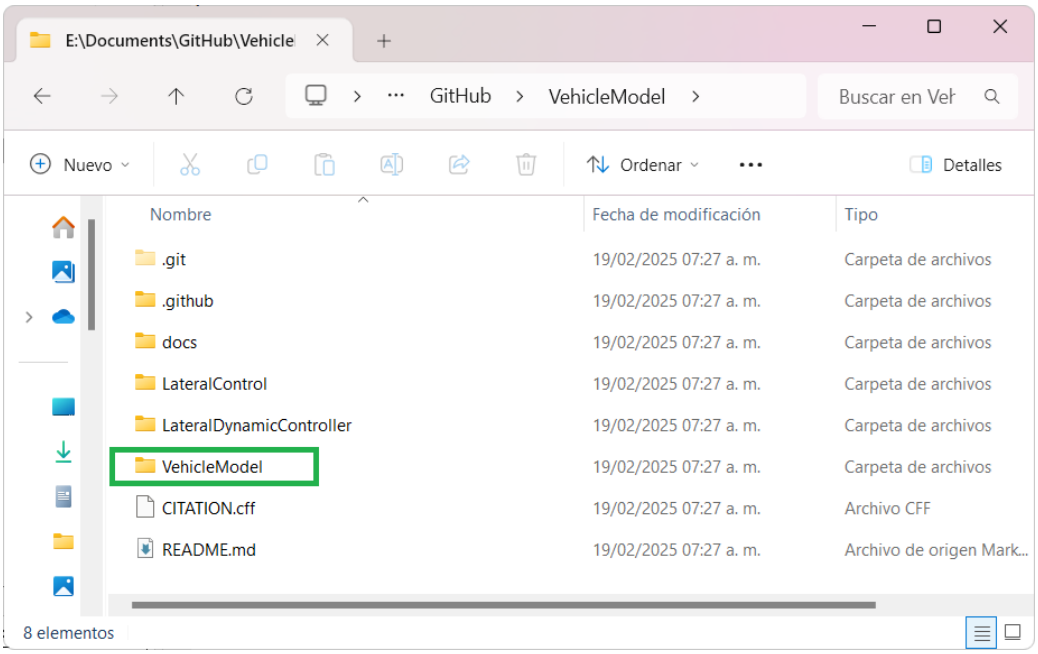

Figure 2: Root folder in the repository.

important file is **LongLatDriveline.slx** in a), since this is the main simulation model where our virtual vehicle is running. The file **VehicleData.sldd** in Fig. 3 b) is a Simulink data file that contains all the parameters we can adjust, according to the characteristics that we want for our vehicle. Finally, the file **waypoints.mat** contains the route that our vehicle will follow; this route can be modified, but we will see how to do that, further in the document.

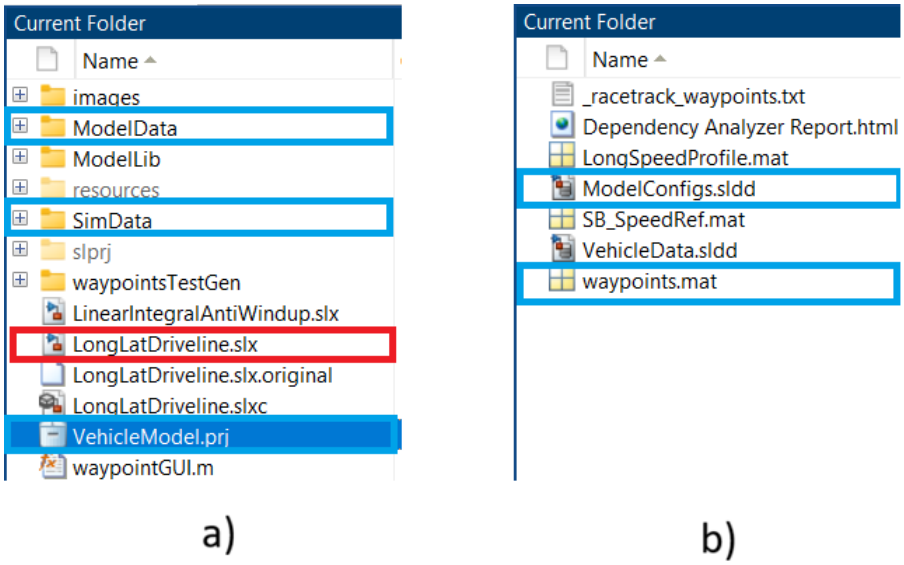

Figure 3: Main working files for simulation. a) folder ./vehicleModel/. b) folder ./vehicleModel/ModelData

Open Matlab and select **VehicleModel** as working directory as in Fig. 4 and double click the file **VehicleModel.prj** which is the Simulink project file. Automatically the model **LongLatDriveline.slx** will be loaded.

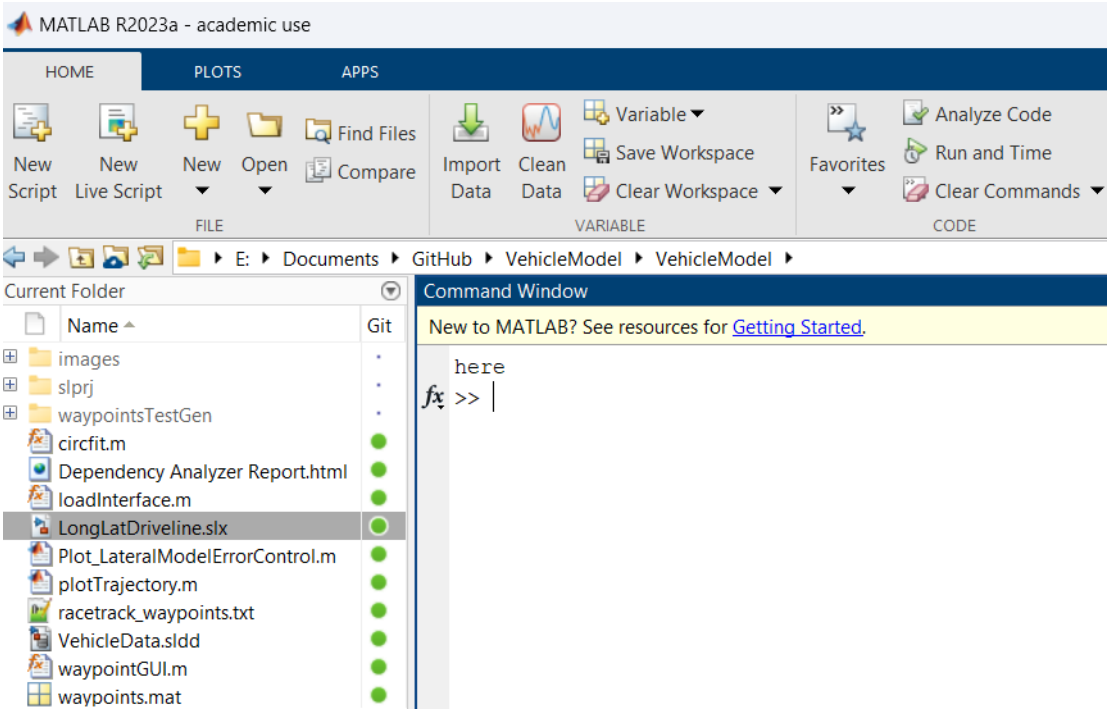

Figure 4: Working folder in matlab.

When the Simulink model is open, a window called *Initial conditions* will appear, as shown in Fig. 5. This window will contain, by default, the initial position and initial orientation detected in the way-point file **waypoints.mat** located in the root directory. If you want to use this initial state of the vehicle, just click in "Accept". However, you can change it. It is recommended not to change them too far from the detected position because this might cause the lateral controller get lost and never find the route. If you don't want to change any previous initial conditions setting, just close the window. The opened model will have the appearance as shown in Fig. 6.

The Model is divided into two sections. The upper part corresponds to the longitudinal dynamics, and the lower part is the lateral dynamics.

Now, let us verify that we have all the settings needed for our first run. For this first run, we will not use any electric motor as actuator or power element. So, let us verify that we have the simplest configuration.

First, in the Simulink window, locate the "*Model Settings*". A common place where to locate is

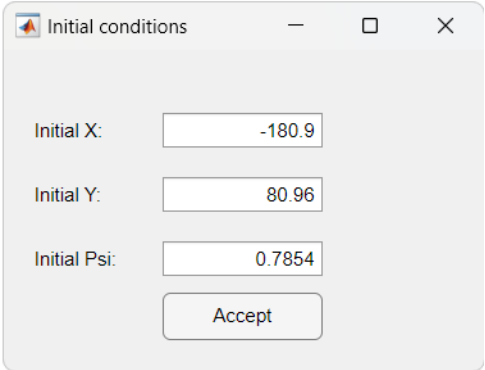

Figure 5: Initial position and orientation of the vehicle for simulation

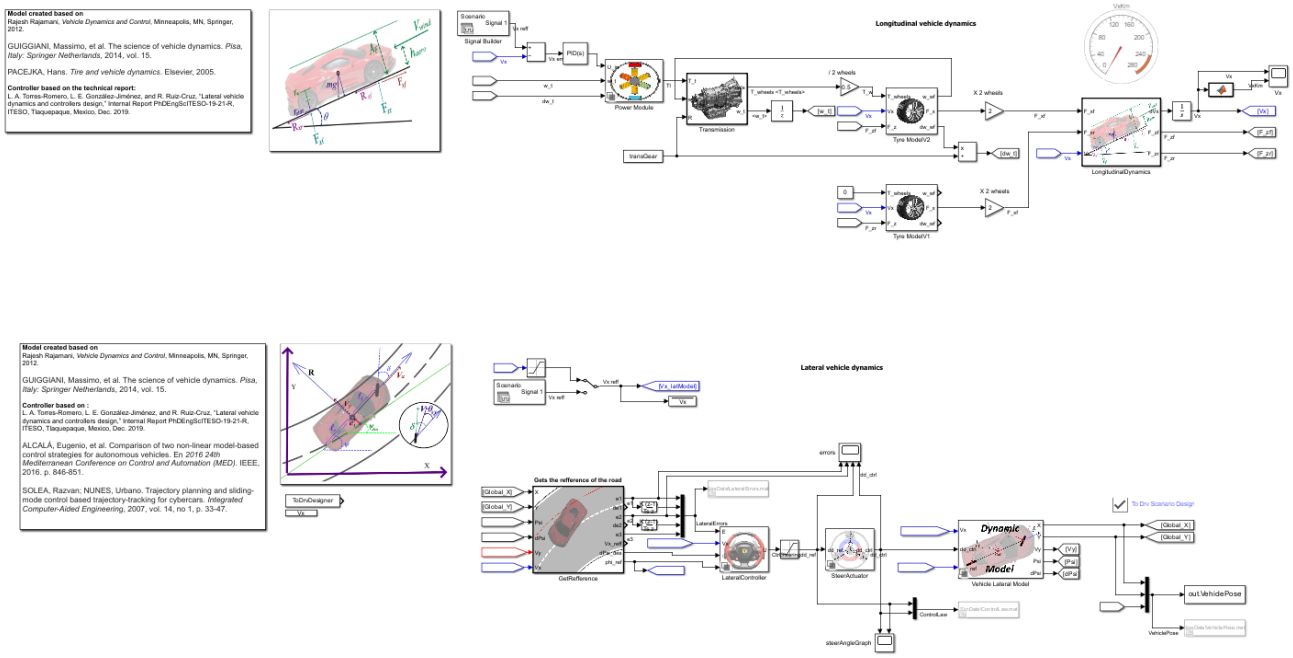

Figure 6: Simulink main model.

shown in Fig. 7.

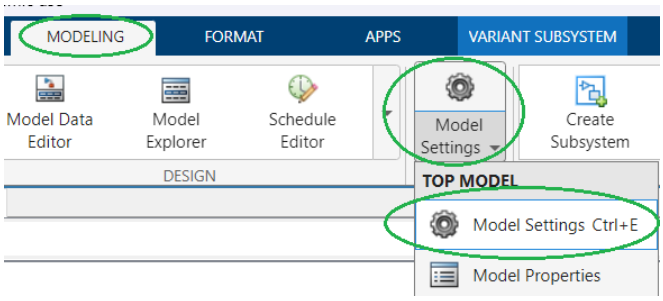

Figure 7: Model settings location.

Now, be sure that the settings are as follows in Fig. 8. The important configurations are in the

solver section. *Type* = ”Fixed-step”, *Solver* = ”Auto”, and *Fixed-step size* = 0.002

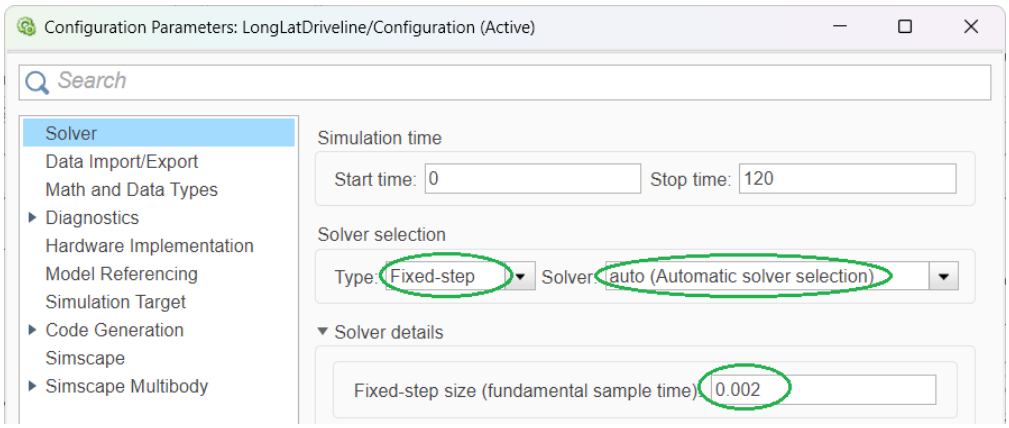

Figure 8: Model settings.

now, let’s verify the variant blocks configurations. For longitudinal dynamics be sure that the power module block is *Direct signal (Signal builder)* and for the lateral dynamics, the lateral controller block, the controller *Ackermann error feedback* control is selected, for the actuator block, *No actuator* shall be selected, and for the LateralDynamics block, *simplified model* shall be selected. Figs. 9 and 10 show these blocks.

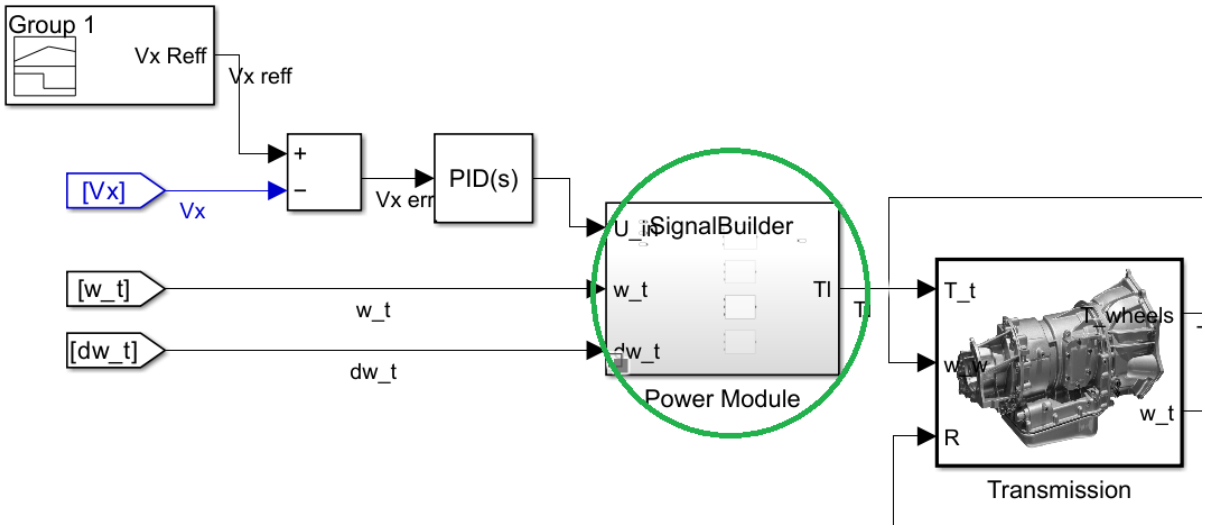

Figure 9: Power block.

In this point, we have everything ready. Just click run and wait until the simulation is complete. When the simulation ends, if the vehicle completed the task, a pop up window with the message ”You have arrived” appears, and a window with the trajectory appears too as in Fig. 11. Figs. 12 and 13 show an example of the data that you can get from the simulation. for example, how the vehicle accelerates, or

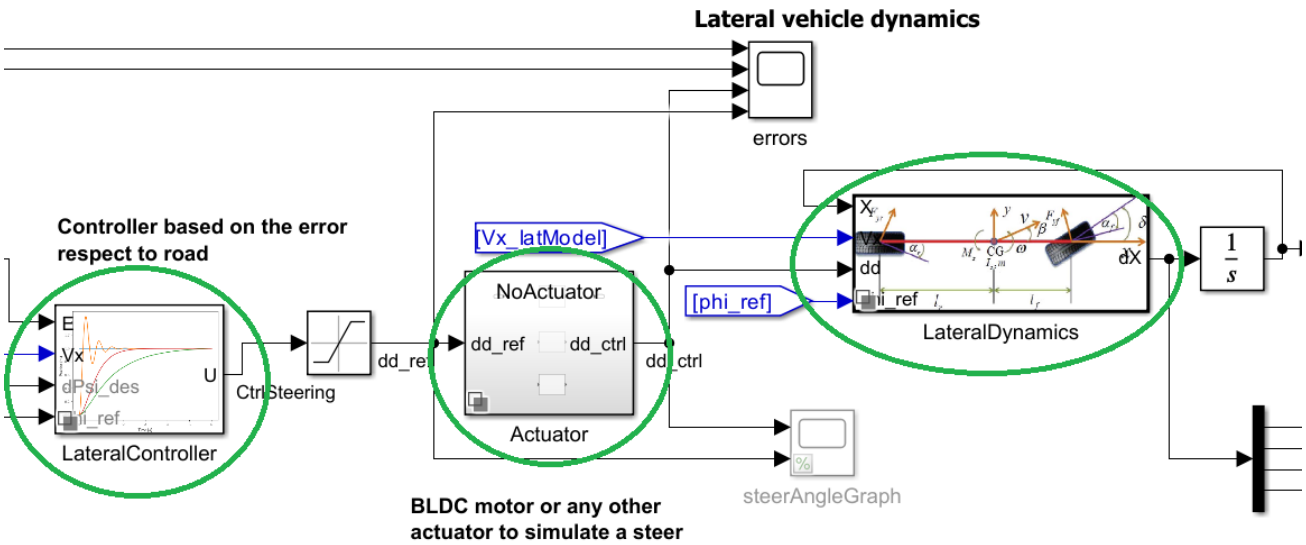

Figure 10: Lateral blocks selection.

the errors with respect to the road; in this case, Fig. 13 shows e1 and e2 that correspond to the lateral error and orientation error, respectively. From here, your next steps could be running several times this simulation, putting several scopes throughout the model to see the behavior of the dynamics. You can explore further the model, change dynamic blocks, actuators, controllers, etc. In a later chapter you can see how to do so. It is required a brief explanation to modify these elements because you will need to modify some configurations of the simulation in order to get good results, otherwise you might get errors or incorrect results; however, in this quick start you have the basic to start up.

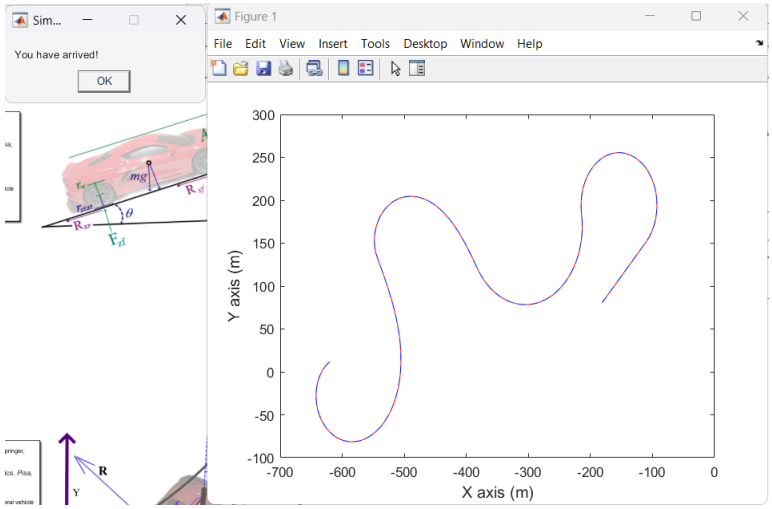

Figure 11: Trajectory and message of completed task at the end of the simulation.

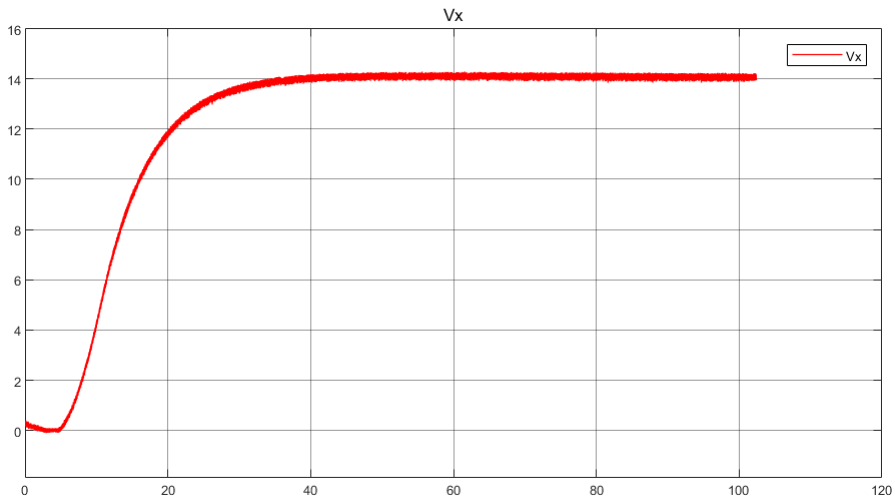

Figure 12: Longitudinal velocity profile.

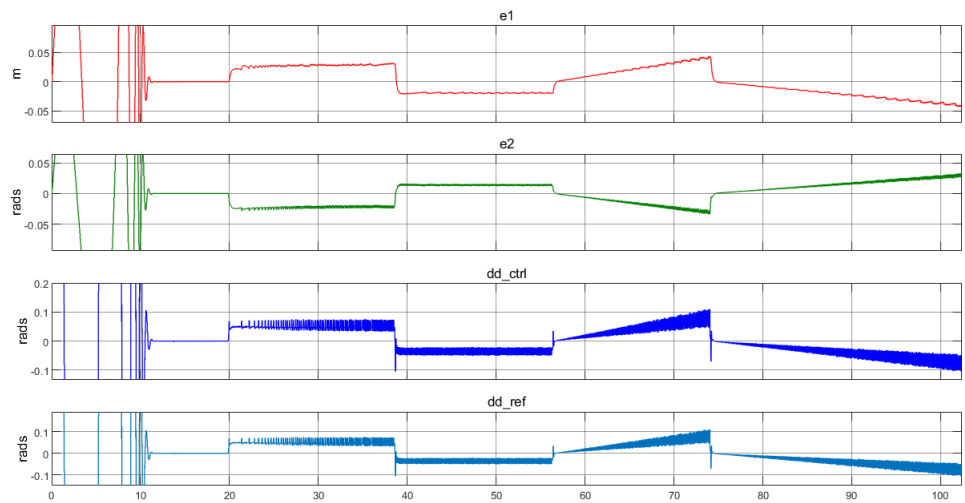

Figure 13: Lateral errors and response of the actuator.

III. LATERAL DYNAMICS SECTION

Now, that you could run once the model following the quick start section, let us take a look at the elements of the lateral dynamics section. Fig. 14 shows the elements of the lateral section. Elements marked in blue with numbers are the principal elements, whereas the blocks marked with letters in yellow circles are auxiliary elements.

First, let us explain the principal elements:

1. **GetRefference block**. This element takes the way-points stored in the variable *racetrackwaypoints* and calculates, based on the current state of the vehicle, lateral error **e1**, orientation error **e2**, and

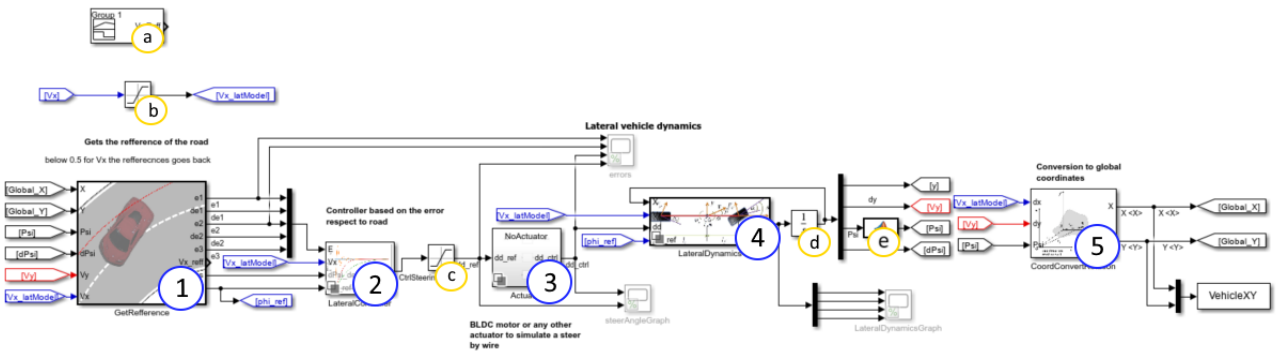

Figure 14: Lateral dynamics elements.

their derivatives **de1**, and **de2**. **Vx_reff** is the reference speed defined in the way-points variable; this is used as a reference control in the longitudinal section. **dPsi_des** is the desired calculated orientation, based on the way-points. **phi_ref** is the superelevation of the road; this is calculated based on a generic and simplistic equation for road design; be aware of this if the superelevation feature is relevant for you.

2. **LateralController block**. This is a Simulink variant block. Here, different lateral controllers are implemented. You only have to choose your desired controller by selecting the variant. Here, is where you can create your own controller version.

3. **Actuator block**. As the same for the lateral controller, this is a Simulink variant where you can choose or implement your own actuator, such as an electric motor which will keep the steering angle commanded by the lateral controller block.

4. **LateralDynamics bolck**. This is a variant block with different versions of the lateral vehicle dynamics. You can also choose or create new versions here as well.

5. **CoorConvertRotation block**. This block is in charge to get the current coordinates of the vehicle based in the state variables of the model. These coordinates are a feedback of GetRefference block which is used to calculate the errors against the road.

There are some minor elements that work as auxiliary elements. These elements are identified in Fig. 14 with letters (a to e) within yellow circles.

a. *Signal builder block*. This block is used when you want to provide a specific longitudinal speed to the lateral section. In this case, you might ”comment out” the longitudinal section and connect the *Vx_latModel* label (before the saturation block) to this.

b. *Saturation block*. This is to establish limits to the vehicle speed that feeds the lateral section. Sometimes, some disturbances or over-impulses might come from the longitudinal dynamics and this block will stop this. Another scenario is when, in this current version of the lateral section, is not prepared to receive negative speed (reverse), so, this block will prevent the reverse that will crash the simulation.

c. *Saturation block*. This saturation block after the controller block is to stop over-impulses coming from the LateralController block. Ideally, your control should not reach the limits of the saturation because then that means that you are losing the effects of your control. However, some times the initial over-impulse might cause the vehicle to get lost and never reach the goal. Once your control is capable to reach the end of the trajectory, comment-through this block and make a fine tuning of the parameters of your controller.

d. *Integrator block*. This block integrates the dynamic system. Take care of the initial conditions within this block. The initial conditions are updated at start-up of the model. You can modify these conditions through the window dialog "Initial conditions" or manually in this block.

e. *wrapTo2Pi*. This block limits the value of the orientation angle ψ to keep it within the range $[0, 2\pi]$

A. *Choosing a lateral dynamic controller*

To select a different controller for the lateral dynamic control, just make right click in the lower left corner of the LateralController block, like in the Fig.15 , select *Label Mode Active Choice* and a list with all controllers will appear. Just select the one you desire.

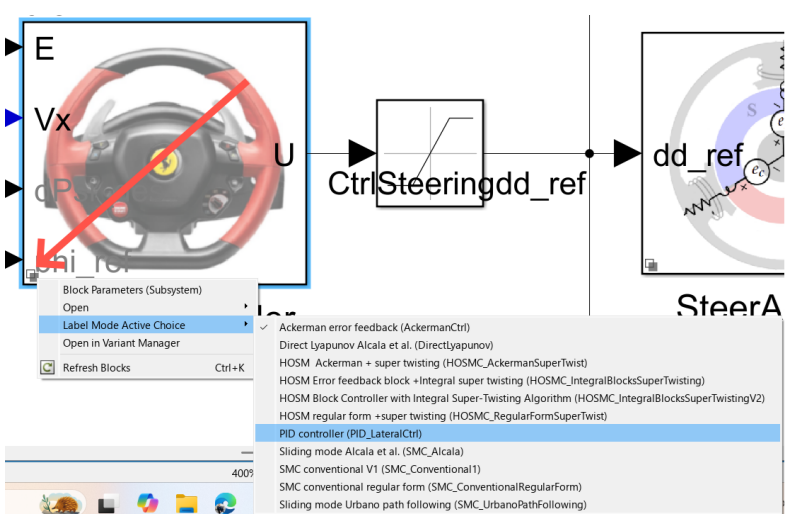

Figure 15: Variant selection of the lateral controller.

Below, in table 1 is a list with a brief description of the controllers.

Table 1: LATERAL CONTROL ALGORITHMS

Name	Description
Ackermann error feedback	This implements the method of Ackermann taken the lateral error model as basis.
Direct Lyapunov Alcala et al.	This implementation is based on [2].
HOSM Ackerman + super twisting	This control eliminates the linear part error using the Ackerman technique and the non linear disturbances using super-twisting
HOSM error feedback block + super twisting	This control eliminates the linear part error using control by blocks technique and the non linear disturbances using super-twisting
HOSM Block controller with integral super-twisting algorithm	This implementation is based on [3]
HOSM Regular form + super-twisting	This implements the integral super-twisting with the conventional smc defined in [4]
PID controller	Simple PID controller.
Sliding mode Alcala et al.	This implementation is based on [2].
SMC conventional V1	This implementation is based on [4].
SMC conventional V2	
Sliding mode Urbano path following	This implementation is based on [5].

B. *Choosing a steering actuator*

The controller almost never actuates alone. There is always a device that is stimulated by the controller and the nature of this device (actuator) introduces disturbances to the system. Here are some actuators that can be chosen for the simulation. See Fig. 16 and table 2.

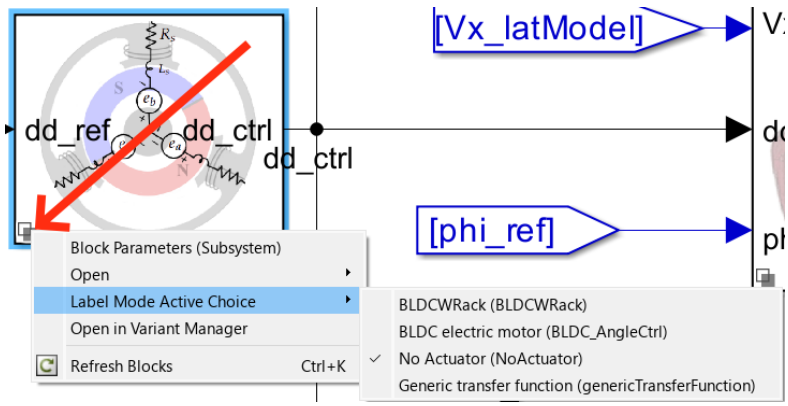

Figure 16: Variant selection of the lateral controller.

Table 2: LATERAL ACTUATORS

Name	Description
BLDC with steering rack.	This implements a BLDC motor with the rack and pinion mechanism for load simulation.
BLDC electric motor.	This the BLDC motor with a constant load.
No actuator	This only bypasses the block to avoid add disturbances.
Generic transfer function	This adds a second order transfer function that emulates the electric motor. This help to add disturbances allowing bigger simulation time steps and more faster simulation.

C. *Choosing a lateral model*

There are several lateral models for election. This can help specially for comparisons of performance. We have two approaches, the dynamical model and the kinematic. The kinematic model is useful for simulations at low speeds, basically below 10km/h while the dynamical model is useful for higher speeds. Fig. 17 and table 3 show the options to choose on these models.

Table 3: LATERAL MAIN MODELS

Name	Description
Dynamic Model.	For high speeds, the forces acting on the vehicle needs to be taken into account. This model is based on the forces.
Kinematic Model.	Model based only on the geometry. This can be useful for speed ¡ 10km/h

Besides the main model selection, if *Dynamic model* is chosen, inside of this block there is another variant block that allows to choose between different versions of the dynamic model. Fig. 18

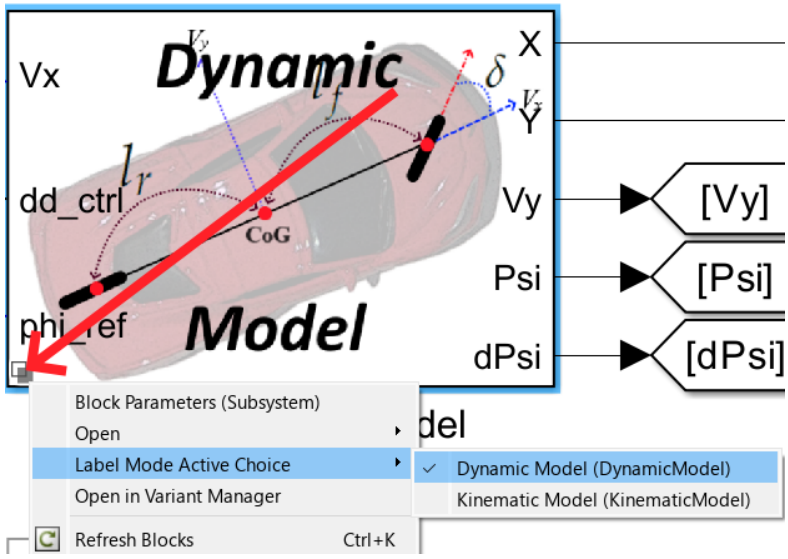

Figure 17: Variant selection between kinematic or dynamic models.

shows the inside of the block with the two variants inside. Inside the dynamic model block, there is another variant with more options. See Figs. 18 and 19, and table 4.

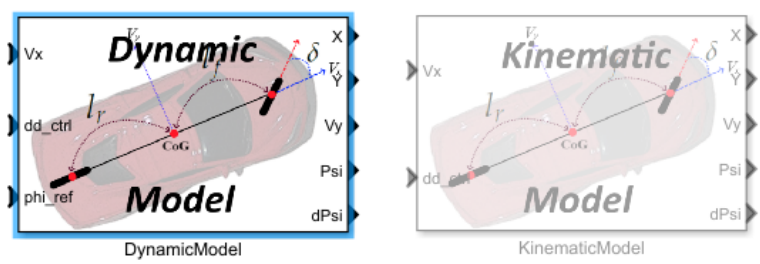

Figure 18: The two variants blocks for the lateral model.

Table 4: LATERAL DYNAMIC VERSIONS

Name	Description
Simplified Model.	This model uses the small angle simplification and considers constant the longitudinal speed.
No Simplified Model.	This model avoids simplifications.
Lateral dynamic with disturbances.	This model is the same as the model not simplified, but it has two sine blocks to add disturbances to the dynamic.

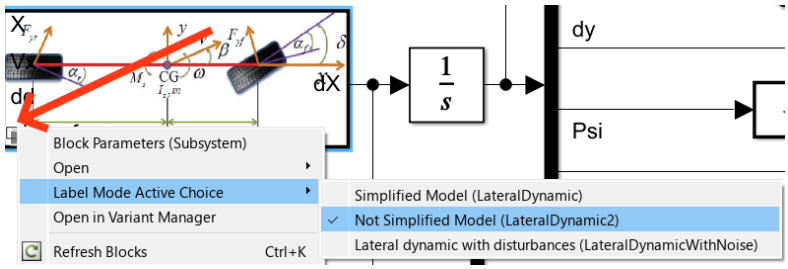

Figure 19: Variant selection for different version of the dynamical model.

IV. LONGITUDINAL DYNAMICS SECTION

Fig. 20 shows the elements of the longitudinal dynamics and the elements are described in the next list.

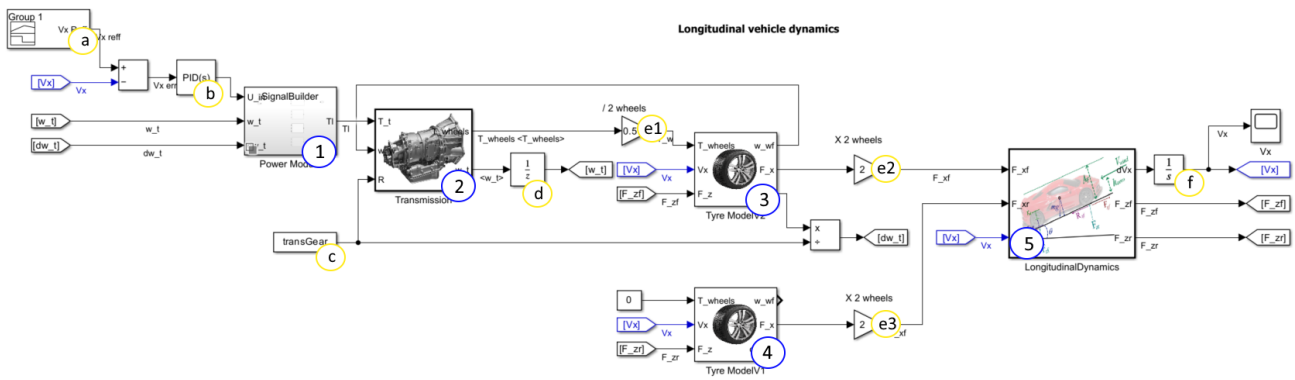

Figure 20: Longitudinal dynamics elements.

1. **Power module**. This is a variant module is intended to define the main source of traction of the vehicle. There is a variant where there is no implementation at all, and it is only a direct connection and then, as in the image, the PID is the source of torque; use this if you want to simulate the dynamic but you are not very interested in the power source. Adding an electric motor makes the simulation slower. The output of this module is **Tl**, which is the torque load delivered by the power source. The inputs **w_t** and **dw_t** are the angular speed and angular acceleration of the gear that connects the power module and the transmission. **U** is the control input.

2. **Transmission**. dynamic of the transmission. Basically converts the input torque to the output torque according the gear relation. The block receives as input **T_t**, **w_w** and **R** which are the input torque coming from the power source, the angular speed of the wheels, and the gear ratio, respectively. The outputs are **T_wheels**, and **w_t** which are the torque delivered to the wheels and

the angular speed of the gear that connects the main power with the transmission.

3. **Frontal tire block**. This is the front tires module. This block represents represents only one tire. that is the reason the input torque is divided by two and the force delivered as output is multiplied by two, to simulate that we have two frontal tires. This is a limitation of the bicycle model.

4. **Rear tire block**. The same as the last one, but it is a different instance to handle different properties versus the frontal tires.

5. **LongitudinalDynamics block**. This block contains the main physic equations of the second Newton's law that are involved to hove the vehicle forward, including the aerodynamics equations.

Now, the auxiliary elements that glue the components are:

a. *Signal builder*. This block creates the speed profile for reference.
b. *PID*. Block to control the main power module.
c. *transGear*. Constant block with the gear ratio.
d. *Delay block*. In order to create an algebraic loop, it is required to add an delay block.
e1. *Gain block*. block that divides the incoming torque between to tires.
e2. *Gain block*. Block that multiplies the force generated by the tire block by two frontal tires.
e3. *Gain block*. Block that multiplies the force generated by the tire block by two rear tires.
f. *Integrator block*. Solves the differential equations of the longitudinal dynamics.

A. *Choosing the Power Module*

Fig. 21 shows the selector of the variant for the power module of the longitudinal dynamics. You can find the SI engine and different versions of electric motors. See table 5.

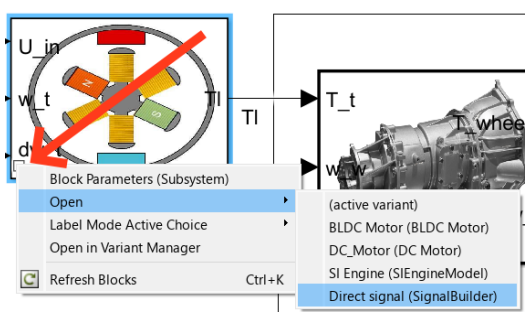

Figure 21: Power module selection.

Table 5 shows the different options.

Table 5: POWER MODULE OF THE LONGITUDINAL DYNAMICS

Name	Description
BLDC Motor.	This is a variant block with several versions of BLDC motors inside (BLDC SVPWM, BLDC abcModel, BLDC dqModel).
DC_Motor.	DC motor version.
SI Engine.	Dynamic model of a spark ignition engine.
Direct signal.	Direct connection to bypass input to output.

V. MODIFYING VEHICLE PARAMETERS

All parameters of the simulations related to the physics of the vehicle are stored in a Matlab database file .sldd. The file you can find it in **/ModelData/VehicleData.sldd** in the project. See Fig. 22 and table 6.

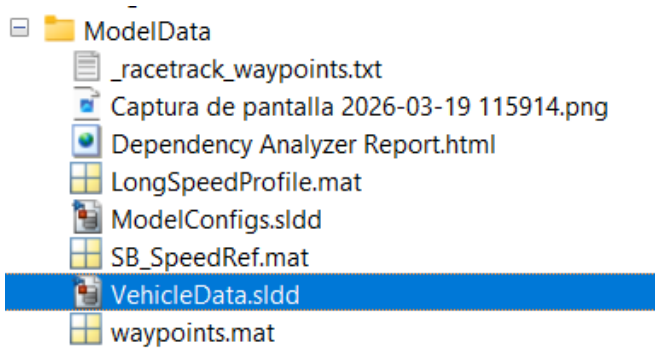

Figure 22: Simulation Parameters Location.

Table 6 shows the parameters with their mathematical symbols.

VI. CREATING NEW ROUTES

A. *Introduction*

The Waypoint Generator GUI is a MATLAB-based tool that allows users to create waypoint sequences for different driving test strategies. Users can select predefined strategies, configure parameters, and generate waypoints that can be saved and visualized.

Table 6: MODEL PARAMETERS

MATLAB name	Symbol	Default val
A_F	A_F	2.341157
C_af	C_{af}	80000
C_ar	C_{ar}	80000
C_d	C_d	0.24
InitialPsi	ψ_0	0.78539816339744828
InitialX	X_0	-180.91105760998738
InitialY	Y_0	80.964126937568835
Iz	I_z	2873
Nf	N_f	2
Nr	N_r	2
V_wind	V_{wind}	0
f	f	0.01
g	g	9.81
h_aero	h_{aero}	0.65
h_gc	h_{gc}	0.65
lf	l_f	1.1
lr	l_r	1.58
m	m	2238.932
r_stat	r_{stat}	0.342
r_w	r_w	0.352
rho	ρ	1.225
theta	θ	0
transGear	R	0.1027

B. *Getting Started*

1. **Launch the GUI:**
   - Run the `waypointGUI` function in MATLAB to open the interface.
2. **Select a Test Strategy:**
   - Choose a strategy from the dropdown menu labeled **"Select Strategy"**.
   - Available strategies:
     - `StreightLineTest`
     - `ConstantSpeedVariableRadiusTest`
     - `ConstantSteeringWheelAngleTest`
     - `VariableSteeringWheelAngleTest`
3. **Add a Strategy:**
   - Click the **"Add"** button to configure and add the selected strategy.
   - If the strategy requires parameters, a dialog box will appear to input values such as distance, speed, radius increment, or steering angle.

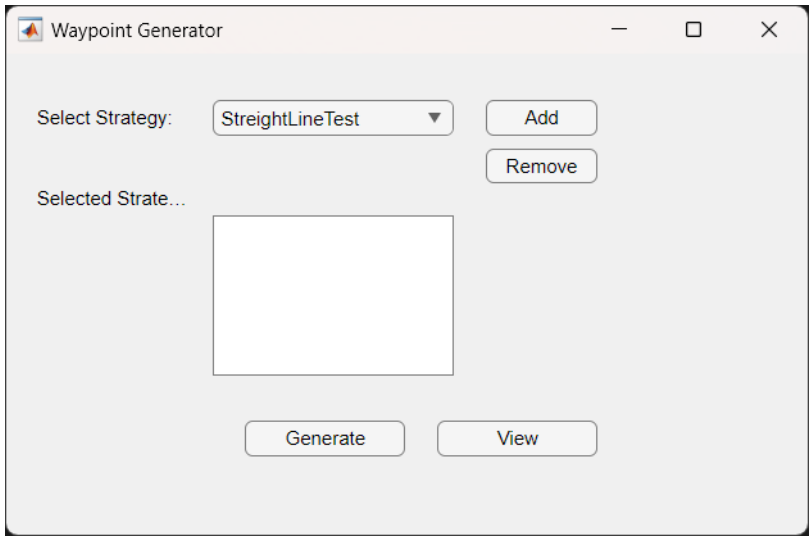

Figure 23: GUI to create new way-point files.

- Click **"OK"** to confirm and add the strategy to the list.

4. **Remove a Strategy:**
   - Select a strategy from the **"Selected Strategies"** list.
   - Click the **"Remove"** button to delete it.
5. **Generate Waypoints:**
   - Click the **"Generate"** button to create waypoints based on the selected strategies.
   - The waypoints are saved in a file named `waypoints.mat`.
   - A success message will appear once the waypoints are saved.
6. **View Waypoints:**
   - Click the **"View"** button to visualize the generated waypoints on a 2D plot.
   - The waypoints will be displayed as a path with X and Y coordinates.

C. *Closing the Application*

- Closing the GUI will automatically clean up and remove any added paths related to the tool.

D. *Additional Information*

- The waypoints are generated using the `GenerateWaypoints` function, which applies the selected strategy to create a continuous path.

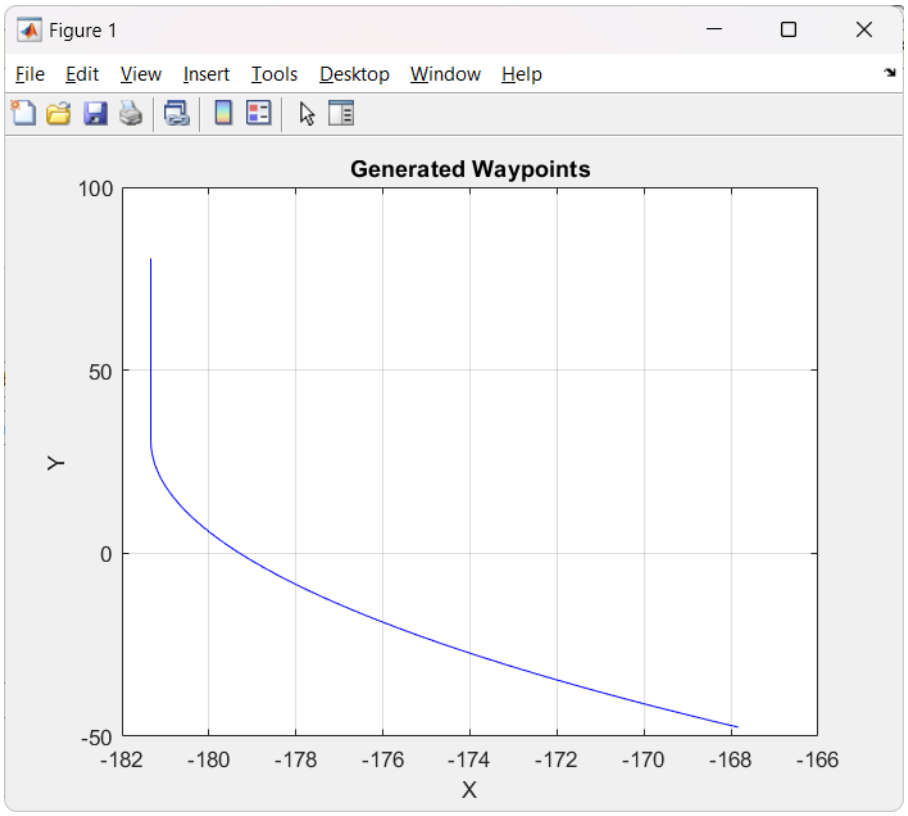

Figure 24: GUI to create new way-point files.

- Ensure all required files are in the working directory before running the GUI.

  For further modifications or custom strategies, users can edit the MATLAB scripts accordingly.

VII. CARLA INTEGRATION

This feature is still in progress.

REFERENCES

[1] L. A. LUIS ARTURO Torres-Romero, "VEHICLE DYNAMICS AND CONTROL SIMULATION SYSTEM." [Online]. Available: https://github.com/L-Arturo-Torres-Romero/VehicleModel

[2] E. Alcala, L. Sellart, V. Puig, J. Quevedo, J. Saludes, D. Vázquez, and A. López, "Comparison of two non-linear model-based control strategies for autonomous vehicles," in *Proceedings of the 24th Mediterranean Conference on Control and Automation (MED)*, Athens, Greece, 6 2016, pp. 846–851.

[3] L. A. Torres-Romero and L. E. González-Jiménez, "A block controller with integral super-twisting algorithm for the path following of a self-driving electric vehicle considering actuator dynamics," *World Electric Vehicle Journal*, vol. 16, no. 12, p. 643, 2025.

[4] L. Torres-Romero, R. Ruiz-Cruz, and L. González-Jiménez, "Path-following sliding mode controller for an electric vehicle considering actuator dynamics," *Machines*, vol. 12, p. 219, 2024. [Online]. Available: https://doi.org/10.3390/machines12040219

[5] R. Solea and U. Nunes, "Trajectory planning and sliding-mode control based trajectory-tracking for cybercars," *Integrated Computer-Aided Engineering*, vol. 14, pp. 33–47, 2007.

VIII. TABLES